RESEARCH REPORT

LLM agents that talk to each other about traffic

Prior art, control-loop engineering and a measured simulator decision for a traffic-signal simulation with per-intersection agents and a gridlock orchestrator.

Compiled 2026-10-30 · **Contents** three research dossiers, a recommended arc, 42 references
Machine-checked on: arXiv API (2026-10-29), SUMO 1.28 + libsumo, GitHub API

Contents

Part A — Prior art2
What the field already does2
Part B — Control loop8
Making the model reliable and fast8
Part C — Simulator decision14
Part D — Recommended arc21
References22

Part A — Prior art

What the field already does

Status: citation-grade survey. Resetting note: the task's `IDEA.md` was for the *adjacent* (road-arc) design. This dossier is written against the **node-semantics** design:

- Per node: Each signalised **node** carries the agent. Each node's agent only observes the **arcs** that touch it, i.e. the **road arcs** that either enter (upstream) or leave (downstream) the node.
- Each node's agent sees "Pressure" (pressure per node) as: `p(a) = max(0, up(a) − down(a))` on movement `a` (upstream queue minus downstream queue, floored at 0 against the conceptual pressure), maximised over movements in a candidate phase. Pressure is used to choose a phase and decide **hold vs switch**.
- Node agents **communicate** with neighbour node agents (the arcs that connect them).
- An **Orchestrator LLM** detects gridlock and **overrides** the subagents; it keeps control of the **affected nodes** until resolution, then **hands control back**.

All arXiv counts below are machine-checked against the arXiv API on **2026-10-29** (queries in `research/_arxiv_raw.json`). Abstracts for all IDs below are pulled from the API and stored in `research/_abs.json`; full texts read live in `research/_pdf/` and `research/_fulltext/`.

0. Landscape counts (measured, not guessed)

Query (arXiv `all:`)	Hits	Reading
`"traffic signal control" AND "large language model"`	**26**	small but real; ~20 directly on-topic (the rest are RL/adjacent)
`"traffic signal" AND "LLM agent"`	5	tight core
`"traffic signal control" AND "multi-agent" AND "large language model"`	7	core
`"max pressure" AND "large language model"`	**0**	nobody has bridged these two vocabularies
`"traffic signal" AND deadlock`	**2**	gridlock is nearly absent from this literature; neither hit is an LLM paper
`"gridlock" AND "traffic signal"`	**2** (both: OverFlowLight 2606.27381, auction metering 2508.09678)	—
`"traffic signal control" AND orchestrator`	**2**	neither is the described architecture

The absolute literature is small. That cuts both ways: there is room, but the *shape* of the design is heavily pre-figured — see §1.

1. LLM / multi-agent-LLM TSC papers, newest first

Each entry: mechanism in 3–5 lines, **LLM control** (cadence / model / latency handling), and **distance to the design**.

1.1 EVOCATION OF THE ORCHESTRATOR PATTERN: SCORED

#	Paper	Per-node agent?	Agents talk?	Supervisor preempts locals?	**Verdict**
1	HiLLTS (2607.22691)	**No** — lower layer is a *rule* controller	No	City LLM issues directives that "override the base policy"	**~65% of the design.** Missing: per-node LLM, node–node talk, gridlock-specific trigger, hand-back
2	CoLLMLight (2503.11739)	**Yes** (one agent per intersection)	Only implicitly (neighbour occupancy baked into the prompt)	No	~50 %. Its own conclusion names a "regional-level planning" tier as future work
3	Virtual Traffic Police (2601.15816)	No — single intersection	No	Upper LLM **is** the preemption layer; tunes MP weights	~45 %, but the *preempt-then-restore* framing is explicitly human-police
4	TrafficGen (Frontiers AI 2026)	"IntersectionAgent(s)"	Via a Supervisor *router*	Supervisor routes to emergency/intersection agents	~55 % architecturally, but **single-intersection text twin**, deterministic, no gridlock
5	Traffic-R1 (ACL 2026)	Yes	**Yes — explicit async message passing**	No	~45 %; the comms layer is the closest match, the supervisor is absent
–	*Design under review*	Yes	Yes	Yes, gridlock-triggered, with hand-back	—

(This table is the whole competitive picture: no published work closes all four columns.)

(Scores are my own reading of the abstracts/full texts, not the authors' claims.)

Be brutal: the orchestrator pattern is *not* novel in the abstract — two-tier human/manager-override is the classic traffic-management structure, and three 2026 papers (HiLLTS, Virtual Traffic Police, TrafficGen) each ship a version of it. What nobody has is the *specific* conjunction: LLM per node **and** node-to-node a2a messages **and** a supervisor that keys on **gridlock** and **hands control back**. That conjunction is the only defensible claim, and §5 says how fragile it is.

1.2 PAPERS — NEWEST FIRST

2609.22746 — ProcessLight: Process Supervision for LLM-Based TSC. EMNLP 2026. - Mechanism: decomposes each signal decision into verifiable **semantic reasoning steps**; trains with step-wise credit assignment (STeP-PO) over a step tree, scoring each step's quality and influence on the final action. - LLM control: one LLM call per phase decision; model is a fine-tuned backbone (not specified in abstract); no latency handling. - Distance: single-intersection, per-node agent, no comms, no supervisor. Competes on *reasoning quality*, so a shiny LLM-per-node pipeline must beat it on a metric that isn't only AWT.

2608.18263 — SIGMA. SUMO, multi-faced RL + LLM translating emergency instructions into priority vectors; four Kolkata intersections. Explores **graceful degradation under LLM failure** — useful to copy. No comms, no supervisor.

2604.25259 — DGLight (thesis). DQN (CoLight-style critic) scores LLM candidate actions; GRPO fine-tune; Jinan/Hangzhou. LLM call per decision; no latency handling. Pure single-intersection.

2604.23902 — LLM-Augmented TSC with LSTM prediction. LLM **evaluates candidate actions** from an LSTM predictive controller; a **safety filter** validates every output — "LLMs as constrained reasoning and decision-support rather than direct low-level controllers" is its stated thesis. SUMO; balanced/peak/surge demand.

2604.05663 — CuraLight. IJCNN 2026. RL agent collects trajectories → imitation fine-tune of an LLM; multi-LLM **debate ensemble** scores candidates for preference supervision. Training-time ensemble, inference-time single model.

2604.05535 — SignalClaw. Evolved, interpreter-checkable skill code; an event detector + priority dispatcher composes skills at runtime for EV/incident. Reported delay 7.8–9.2 s; EV delay 11.2–18.5 s vs 42–95 s for MP/DQN.

2604.17456 — TrafficClaw. Single LLM agent for multiple urban subsystems (signals, freeway, transit, taxi) with persistent memory and agentic RL. **Not** per-node — the agent is the controller.

2601.15816 — Virtual Traffic Police. NUS, Elsevier preprint. Upper LLM tunes **max-pressure saturation weights** and MPC capacity parameters in response to incidents; RAG over a "traffic language database" + LLM verifier that self-refines the retrieval store. - LLM control: invoked every **Q** steps (Q is a free parameter; the paper's algorithm gated on `s mod Q = 0`); DeepSeek-V3-671B, GPT-4o; temperature 1 for the generator, 0 for the verifier. **No latency measurement at all.** - Latency stance: argues LLM inference "may hinder real-time responsiveness" and that phase transitions "demand sub-second-level responsiveness" — so the LLM sits **above** a conventional controller that does the sub-second work. - Distance: single isolated intersection; MP is the lower layer; the orchestrator only tweaks parameters and never takes over phase selection. Its conclusion asks for "large-scale urban networks" — exactly your setting.

2511.19253 — MAESTRO. Moves the LLM *out* of the loop entirely: offline curriculum + reward-function generation for MADDPG. 16 Hangzhou intersections.

2511.00136 — HeraldLight. Dual LLM: Herald forecaster, LLM-Agent, LLM-Critic that repairs hallucinated actions. CityFlow, 224 intersections; "parallel fine-grained" per-intersection agents. No comms, no supervisor.

2510.26242 — REG-TSC. Reviewer-RAG + emergency-aware deep reasoning; distributed per-intersection agents, ~**one LLM call per intersection per simulation step**; Llama-3.1-8B with LoRA; SUMO, 17–177 heterogeneous intersections. Deep reasoning only activates for emergency nodes (a cheap complexity gate). No comms, no supervisor.

2508.02344 — Traffic-R1. ACL 2026 (aclanthology 2026.acl-long.995), HKUST-GZ + PCITECH. Qwen2.5-3B, two-stage RL. **Explicit asynchronous message-passing network** between neighbouring intersection agents (claims synchronized parallel workflows are either inefficient or incompatible). Deployed on a **single T4-class (Tesla T40)** for inference; 15 s green / 3 s amber / 2 s all-red in CityFlow; deployed on 55k drivers/day with ≥5 % queue reduction. - This is the closest thing to your agent-to-agent layer. The gap: comms carry *messages* but there is no supervisory tier, no gridlock trigger, no hand-back.

2503.11739 — CoLLMLight. HKUST-GZ. Per-intersection agents; a **spatiotemporal graph of neighbouring intersections** (incl. occupied neighbour lanes, occupancy > α=0.5) rides in the prompt; complexity-aware reasoning picks no-coop/simple/complex modes by counting congested neighbour lanes (n_c). Fine-tuned on Llama-3.1-8B. - LLM control: **one LLM call per intersection per decision step**; decision step = 30 s green + 3 s amber + 2 s all-red in CityFlow. Total inference-time axis is a headline axis in the ablation (2.5k–10k s scale). No concurrency or staleness handling. - Should be the primary baseline and threat. Its related work explicitly dismisses "iterative inter-agent communication" as too slow for real time — the closest published argument to your a2a claim.

2407.06025 — iLLM-TSC. LLM **audits** RL actions and overrides implausible ones; −17.5 % AWT under degraded comms. The override pattern at single-intersection scale.

2406.13693 — From Single Agent to Multi-Agent. Small step-up study; fine-tuned LLM gains are "small".

2403.08337 — LLM-Assisted Light. LLM at the centre with perception/decision tools; −20.4 % AWT under sensor outage.

2312.16044 — LLMLight + LightGPT. The origin: prompt-as-state, LLM as controller, distilled LightGPT. 10 datasets, 15 human raters, claims beat 10 frontier LLMs. **Read the prompt design** — senior traffic engineers judge its outputs as the reference for explainability.

2401.00211 — Open-TI. Assistant-to-agent traffic. Operator loop: conversational LLM shells out to simulator training, with a separate agentChatAgent loop — plus some sort of training signal end-to-end.

1.3 NOT LLM, BUT THE RIGHT NON-LLM COORDINATION BASELINES

- **MA2C** (arXiv 1903.04527): localised MARL where each agent's state = own + neighbour fingerprint. The canonical "local agents, local observation" RL baseline.
- **FRAP** (AAAI 2019) and **CosLight** (KDD 2024): phase-level interaction networks and *learned* collaborator selection — the strongest non-LLM evidence for the "agents should talk" premise.
- **UrbanLLM (2406.12360):** general hierarchical manager → specialized expert agent pattern, high-level.

2. Max-pressure theory

2.1 VARAIYA 2013 — THE STABILITY RESULT

Varaiya, P. (2013). *Max pressure control of a network of signalized intersections*. **Transportation Research Part C 36, 177–195**. DOI **10.1016/j.trc.2013.08.014**.

Model: store-and-forward point queues, infinite storage, **fixed-known turn ratios** r(l,m), saturation flows c(l,m). A phase is a single movement (l,m) from input link l to output link m. **Definition (21) of the paper**: the *weight* of a phase is

```
w(l,m)(X) = x(l,m) − Σ_{p∈Out(m)} r(m,p)·x(m,p)
```

i.e. the upstream queue on the phase minus the average downstream queue on m. The **pressure** of a control matrix S is the saturation flow times the phase weight:

```
c(S)(X) = Σ_{(l,m): S(l,m)=1} c(l,m)·w(l,m)(X)
```

and **MP control is the arg-max over control matrices** (eq. 22–23). The paper states (p. 186) verbatim: "The pressure of a signal control matrix S is the sum of the saturation flows multiplied by the weights of all the phases that S actuates simultaneously." **Equivalently** (Levin 2023): `w_ij = x_ij − Σ_k x_jk p̄_jk`, `s* ∈ argmax Σ w_ij Q_ij s_ij`.

Theorem 2: MP is stabilizing whenever the mean demand lies in the interior D° of the feasible demand region; likewise Theorem 4 for adaptive MP with converging parameter estimates; Theorem 5 for weighted queues.

Caveats Varaiya himself records:

- **Requires the mean turn ratios and saturation flows.** (Theorem 2 assumes known; "MP does require knowledge of turn ratios and saturation flows".) Practically this is a fixed turning-movement table — a bad fit for a node agent that only sees raw link counts unless you estimate on-line (→ AMP, Theorem 4, needs convergence).
- **The projected variant** `u_sat` (forbid movements into a saturated link) is **not stable** — gridlock of a different kind (Varaiya's Example 3: all movements blocked).
- **Min-green / weighted-queue variants exist (Thm 5)** to prevent starvation; Lämmer–Treiber and the Transit-MP / mTransit-MP line (2511.00309) document starvation empirically from the CV-sensing side.
- **Saturation-flow misses ("the saturation rate of the lane drops to 0")** are exactly what the LLM in Virtual Traffic Police rewrites by hand.

2.2 HOW CLOSE IS THE DERIVATIVE LITERATURE?

- **PressLight** (Wei et al., KDD 2019, DOI 10.1145/3292500.3330949): reward = pressure, state = lane counts. Single-agent per intersection, no comms. Its abstract claims a theoretical ε-optimality argument but the implemented controller is a one-hop RL policy — the stability theorem does **not** carry over.
- **CoLight** (Wei et al., CIKM 2019, DOI 10.1145/3357384.3357972): graph-attention over neighbours — city, queued neighbour embeddings; reward = queue, not a stability guarantee.
- **MPLight** (Chen et al., AAAI 2020): pressure reward + FRAP-style attention; "all summed queue lengths ≤ some explicit constraint"; again heuristic.
- **Efficient-MP / 2112.02336** and **Advanced-MP / 2112.10107:** restate exact pressure; the Advanced version adds running-vehicle count to the downstream side. If your "pressure" is summed with a `max(0, ·)` you are in Efficient/Projected territory.
- **C-MP (2407.01421):** Weighted downstream platoon emphasis, provable queue stability; the closest "decentralised yet coordinated" MP (stability maintained via Lyapunov analysis).
- **Levin 2023 summary (J. Transp. Eng. A 149(4), 03123001):** best single methodological reference, with the list of where MP theory's assumptions break under finite storage and work-conservation.
- **Decision timescale (2610.32280):** proves a load-dependent critical decision interval — essential reading because the LLM's per-call cadence is a decision interval, and this gives you the theory for *why* the orchestrator can't run at every step.

3. Gridlock / deadlock — the weakest-covered and most differentiating part of the design

3.1 WHAT ACTUALLY CAUSES GRIDLOCK

Gridlock is a **cycle of blocked links**: every arc in a cycle is occupied to capacity, so no node in the cycle can move vehicles, so no node in the cycle empties. In a mesoscopic queue model it needs three ingredients:

1. **Finite storage** (link capacity L(m)). Point-queue models without storage can never gridlock — this is exactly why the MP theorems are blind to it (Varaiya 2013, §4.5: "the model does not permit saturation … This is a limitation" and Fig. 6's conjecture that a network with a loop "can always be subject to gridlock no matter how large the links are").
2. **A demand/route topology that closes loops** — a one-way ring, a rotary, or a ramp/gore with a U-turn path.
3. **A work-conserving green** that keeps feeding a full downstream — this is *the node agent's own choice*. The MP `u_sat` variant fails for exactly this reason: a strict "skip movements into a saturated link" rule stalls the whole cycle.

A point-queue subtlety worth citing verbatim in any write-up: Varaiya's Fig. 6 shows **the identical junction on the left gridlocks, the identical one on the right doesn't** — topology alone decides whether a given blocked-queue policy gridlocks.

3.2 THE SPARSE GRIDLOCK-CONTROL LITERATURE

- **OverFlowLight (2606.27381)** is the only gridlock paper in the LLM-adjacent space. Detector/segment widgets flag queue overflow onto upstream segments, and the controller inserts a **dedicated "overflow phase"** to flush the blocking movement; a rule layer hard-masks unsafe actions before the RL/Q-network output (`a* = argmax_{a∈A_safe} Q`). Real deployment on 43 intersections, −60.4 % overflow events, +18.2 % throughput, 106 ms end-to-end. **It is *intra*-intersection and rule-based — no LLM, no multi-intersection takeover, no hand-back.**
- **Lämmer, Treiber & Rausch 2013** — inflow-regulating green that never feeds more than a link can hold. The direct intellectual ancestor of the user's "Orchestrator restriction". Checkout-free self-healing variants include the *self-healing road networks* work of these

authors. Adjacent: **Tsitsokas & Kouvelas 2022/23** (MFD perimeter control + MP two-layer). **Perimeter control** (via MFD) and **auction metering** (2508.09678) are the macro cousins; neither is agent-based nor LLM.

- **Gate/metering with forbidden movements:** the standard real-world remedy is forbidding the closing turns and holding the gate approach. All formulations below assume a phase *insert*. Neither is a per-node LLM.
- **Regional v. dynamic containment:** none of the above does **hand-back with a state guarantee**. The closest is OverFlowLight's "remove the overflow phase when the queue clears".

3.3 MECHANISM NOTE — WHAT THE IMPLEMENTATION MUST ACTUALLY DO

In a meso simulator, gridlock is *never detected* by a generic engine signal — the sim just freezes and demand piles up behind entry points, and average trip time rises while average speed → 0. So *any* implementation must add: (a) a per-arc full/spillback event, (b) a cycle-adjusted reachability check over the occupied-subgraph on the orchestrator restart, and (c) **an escape hatch** (or a "shrink the cordon by one ring and retry" loop) so one unresolvable movement pair can't leave you in a live-lock loop; (d) a route-follow rule so vehicles don't reverse at dead ends. Budget a day; nobody has published this for an LLM orchestrator and it is where the demo will break first.

Honesty note: arXiv "deadlock + traffic signal" = **2** hits, and neither is about gridlock avoidance. Gridlock in the LLM-TSC ecosystem is essentially unclaimed. That is the single strongest part of the design — and also the part where you must import outside citations (Varaiya §4.5, Lämmer/OverFlowLight) rather than pretend the LLM literature covers it.

4. Benchmarks, metrics, and what a fair comparison requires

4.1 SIMULATORS

- **SUMO** (TraCI). The industry/standards default; every design-deployable study here uses it (Virtual Traffic Police, REG-TSC, all the ×Light family). Microsim + real OSM. Overhead is the *fairness* control for LLM: all baselines and LLM run in the same simulator with the same controls.
- **CityFlow** (KDD 2019, DOI 10.1145/3357384.3358028). Frame-stepable, ~10 000 veh/s; no PyPI wheel — source build only (or the Docker image the docs push). Used by CoLLMLight, HeraldLight, Traffic-R1 and most LLM papers *because* it is an in-sim call loop.
- **RESCO** (NeurIPS 2021 D&B): SUMO + OpenAI-Gym; **but note the paper's own headline result — "previous algorithms are not robust… when a distributed deep Q-learning approach is shown to outperform previously reported state-of-the-art".** Benchmarks are *styled* (grid/CAM/tier/scenario).
- **LibSignal** (arXiv 2211.10649; *Machine Learning* 113:5235–5271, 2024): cross-simulator equivalence across SUMO and CityFlow. **This is the fairness control any cross-simulator claim requires** — the authors' whole point is that models validated in one simulator are not referential to the other.

4.2 METRICS

Standard (all computable directly in either simulator): - **AWT** average waiting time · **ATT** average travel time · **AQ** average queue length · **throughput** (vehicles/h penetrated) · **stops**. - **Fairness CDFs** (p95 waiting time) rather than only means — HiLLTS does this and it is the cheapest way to expose starvation, which an orchestrator *will* cause at a cordon boundary. - **Gridlock metrics** (the design's own claim): overflow events, vehicles-done vs (vehicles-in + left-in-network) deficit, recover time from inject→clear, number of nodes retained in the cordon, and versus a no-orchestrator ablation *and* a rule-only orchestrator ablation.

4.3 WHAT FAIRNESS REQUIRES

1. **Same sim, same phase spec** (30 s green / 3 s amber / 2 s all-red is the Guangzhou/Hangzhou default in 2503.11739/2508.02344/2312.16044). Same vehicle counts and seed runtime.
2. **Update step matched to control frequency** — an LLM on a 1 s step in CityFlow has paid to re-decide 30× per green. Declare it; else you're comparing reasoning policy vs. actuation rate.
3. **Competent contemporary baselines:** CoLLMLight, Traffic-R1 (comms), Virtual Traffic Police (override), OverFlowLight (gridlock mechanism), plus **Varaiya MP** and a *rule-only two-tier* (fixed-time plan + rule overflow phases).
4. **Oversaturation sweep sized from the description**, to make gridlock reachable. the common Jinan/Hangzhou/CityFlow draws are light and rarely gridlock (Varaiya's Example 3 needs *all* links saturated; CoLLMLight's Jinan runs average ~30 veh/min per intersection); scale to v/c > 0.9 on closing links; a gridlock paper with no gridlock in its demand has nothing to show.
5. **Report realized LLM call rate and mean inter-call gap** for every method.
6. **5 seeds, report mean and CI.** A good share of the LLM-TSC papers in §1 report single runs; the means are not reproducible across seeds and you must not inherit that.

4.4 THE TWO CONCRETE TRAPS

- **Node "pressure" with `max(0, ·)` is *efficient/projected* pressure** (2112.02336, 2112.10107). It is exactly as computation-bound as MP and strictly better behaved than plain pressure when a queue runs short, **but Varaiya's Theorem 2 does not cover it** — you may not claim MP stability. Cite Efficient-MP, and call the quantity "projected pressure".
- **Any demand without storage exhaustion never exercises the orchestrator.** *Every* draw must hit a full queue on a closing link at least once, or the demo becomes "LLM agents redo MP". Count and report gridlock/overflow events per run, alongside AWT.

5. Novelty verdict

ALREADY SOLVED (DON'T RESTART, CITE AND BUILD ON)

- **Pressure is a solved cost function.** Varaiya 2013 proves stability when turn ratios and sat-flows are known, and Theorem 4 extends it to converging estimates. Building "pressure + LLM judge over the pressure ranking" is largely a repackaging, and **only proves novelty if you state that the LLM replaces the turn-ratio table** rather than replacing MP.
- **Per-intersection LLM agents.** About 40 % of the 26-hit corpus. LLMLight (2312.16044) is 3 years old; ProcessLight / DGLight / CuraLight are 2026.
- **Agent-to-agent communication.** Traffic-R1's async message-passing is explicit; CoLLMLight's neighbour graph is an implicit form.
- **Supervisor overrides locals.** Virtual Traffic Police (2026), HiLLTS' city layer (2026), TrafficGen's Supervisor (2026), iLLM-TSC's action auditor. A two-tier design is no longer a contribution in isolation.
- **Hold-vs-switch as an action.** MP is `argmax` over phases; hold vs switch is a partition of that action set. Calling it out as a fresh action space is a weak contribution on its own.
- **"LLM call budgets are a problem"** — everyone knows; Virtual Traffic Police and CoLLMLight both argue it. A latency table is table stakes, not a finding.

PARTIALLY SOLVED (AND THE SOURCE OF WHATEVER IS CITABLE)

- **LLM as *preempting* layer that takes control away from controllers rather than advising them.** Virtual Traffic Police is the closest, but it only *tunes parameters* — never takes over phase selection, and works on one intersection. An orchestrator that issues *phase-level* control over a *set of nodes* for a *bounded episode* is new.
- **Control hand-back.** Nothing found. The human "traffic officer takes over, then leaves" narrative is in 2601.15816 and in the OverFlowLight prompt, but no algorithmic hand-back (state contract with the local controller, hysteresis to re-arm, blameless abort) is published.
- **Gridlock as the *trigger*.** OverFlowLight triggers on a queue/spillback *count*, not on a blocked-cycle, and resolves within one intersection. A node-level gridlock detector with phase-override + release is new territory.
- **LLM restricted push grammar.** HeraldLight and others only shape JSON; nobody has published an orchestrator whose action set is restricted to *legal phase intervals/allowed turns at the affected nodes*.
- **Node agents surviving an orchestrator episode** (own-state from stored snapshot, stale replay, resume on hand-back). Not published.

GENUINELY OPEN (IF YOU CAN MOVE HERE, IT'S PUBLISHABLE)

- **Coordinated escape from a joint deadlock** — which node yields, in what order, which movement is sacrificed. Give each node one priority ticket per k steps, the orchestrator verifies the chord is mutually satisfiable, execute in priority order.
- **Bounded-wait / deadlock-freedom.** Operative if you rebuild the action set: give the agent a *zone budget* (call only while Z > 0; Z regenerates at the Green peninsula rate), and the orchestrator is the only process that may take a node below 1. A draining argument is sketchable (the orchestrator is the liveness oracle; agents' budgets make local non-work-conservation bounded), but the arithmetic needs deriving, not hand-waving.
- **Hand-back contract** — hysteresis quota, convergence guard, blameless abort — with measured detection/recovery numbers over a *ledger* of tier: ±0.5 as "within 5 % of no-orchestrator baseline" are concrete.

6. Where the design is exposed (the three attacks a reviewer will make)

1. **"This is CoLLMLight plus a manager"** → answer: per-node pressure at 1–3 Hz versus CoLLMLight's 30 s/phase, plus a bounded orchestrator episode with hand-back; publish the cadence explicitly in the method section, not the appendix.
2. **"The manager is a rule, not an LLM"** → answer: two-tier pressure is not the LLM's contribution; the *coordinated escape order, movement sacrifice, and re-timing during the commit* are. Ablate the LLM (rules-only orchestrator) and show the delta.
3. **"Gridlock is never in your demand"** → answer: two-arc ring + narrow links + a ramp gore + a one-way ring on a segment, from the *first* experiment of the cycle/detection design; report overflow-event counts alongside AWT.

Part B — Control loop

Making the model reliable and fast

Scope: the LLM picks one of 2^k discrete green-phase actions **every 5 s of simulated time** at 16-25 signalised intersections. An orchestrator preempts local agents on gridlock. Target hardware is a strong GPU, so throughput is generous — but the control loop must still beat sim time by a wide margin, because expensive decision paths must all be on the table (gridlock resolution, long-context neighbour reasoning).

The headline design decision (section 3): the LLM NEVER sets the light directly at the 5 s cadence. It sets *modes and parameters* every 15-60 s; a deterministic max-pressure controller with hysteresis, min-green and anti-spillback guards executes every 5 s. Every published system that was actually measured in a simulator says the same thing, and one of them (HiLLTS) reports the LLM-only-per-decision layer doing *better* than a rule baseline while remaining cheap (Table III, arXiv 2607.22691). Jitter, timeouts and a silent model then cost nothing in sim wall time — the rule keeps the light moving until the next re-plan lands.

All numbers below are either quoted from a source or marked **DERIVED** with the arithmetic shown. Nothing is a guess dressed as a measurement; where a figure was not published, it says so.

1. Action-space contracting

1.1 WHY CONTRACTING MATTERS MORE THAN PROMPT TUNING

Traffic signal control (TSC) naturally has a tiny action space — `hold | switch`, or "pick one of 2^k phases" — and that is the whole reason an LLM can be competitive here at all. Two measured facts anchor this section:

- **Raw model output is not parseable by default.** The VLALight paper (arXiv 2609.36934) excludes its own base model, Qwen3.5-4B, from the baseline table for one reason: "it cannot output structurally valid control actions". Post-training was required before that model could drive the sim at all.
- **Structure beats free text in downstream accuracy.** JSONSchemaBench (arXiv 2501.10868) measures constrained decoding on 10k real schemas and finds it *consistently improves* downstream task performance, up to 4% even on GSM8k. Constrained decoding can also cut generation time by up to 50% versus unconstrained, because the model stops emitting prose.

Contract the action into something near-pick-perfect, then *prove at runtime* that the parse is valid before it touches the light.

1.2 THE CONTRACT: 2^K DISCRETE ACTIONS AT 5 S, EXPRESSED AS WORDS THE MODEL ALREADY KNOWS

Each green-phase pair gets a mnemonic name — `NS_T`, `NS_L`, `EW_T`, `EW_L` for through/left of each axis, matching the ETWT/ELWL/NTST/NLSL convention reused across the TSC literature (LLMLight arXiv 2312.16044; CoLLMLight arXiv 2503.11739). At each 5 s tick (iLLM-TSC uses τ = 5 s, arXiv 2407.06025) the model emits **one** of:

```
hold                      # keep current green
switch:NS_T, 15           # request phase change to NS_T, then 15 s green
extend:10                 # keep green, but book 10 more s before re-planning
```

Three realisations of the same contract, in increasing order of decode-time strictness:

1. **Free text with a tiny schema.** `Action{move: Literal["hold","switch","extend"], phase: Phase|None, green_s: Literal[5,10,15,20,30]|None, why: str}` — ~40 output tokens.
2. **`structured_outputs.choice` over a pre-rendered action list.** Only strings are on the token mask, so there is no way to emit a phase the sim would not understand. `choice` is supported in vLLM's OpenAI server as `{"structured_outputs": {"choice": [...]}}` (docs.vllm.ai/en/latest/features/structured_outputs). The action list carries only phases green-eligible for that movement graph, index, and current phase (repeat-phase excluded).
3. **Structural-tag / grammar over a full digest.** Valid JSON, and — importantly — *reachable*, since XGrammar caches context-independent tokens and overlaps mask computation with the GPU, achieving **<40 µs per token** for JSON Schema and CFG on Llama-3.1-8B and up to 100× over prior baselines (arXiv 2411.15100 §4). This talks directly about *token overhead*, which is a different axis from wall-clock (see 1.3).

Green-duration bucket semantics: {5,10,15,20,30 s} is a coarse parameter grid the LLM selects over, NOT the cadence. Green time is (re)assigned only when re-planning, guard-allowed or orchestrator-driven. It also doubles as the handoff to the analytic controller's mode

in section 3.

1.3 THE CONSTRAINED-DECODING STACK AND ITS MEASURED COST

Rank the candidate techniques by what they actually buy at this action space:

Technique	Guarantee	Measured cost (published)
Raw prompt + regex	none	0 overhead, unknown failure rate
Outlines / XGrammar / llguidance via vLLM `structured_outputs`	**Validity by construction** (token mask) — no invalid phase string, no missing field, no malformed JSON	XGrammar <40 µs/token (arxiv 2411.15100; comparisons to ~4,000-9,000 µs/token for prior engines). For cached schemas vLLM reports only "marginally higher" time-per-output-token vs unconstrained; only unique schemas cost real TTFT
lm-format-enforcer	Validity, weaker backend	Shares the 4,000-9,000 µs/token class with Outlines pre-XGrammar (arXiv 2411.15100)
instructor (Pydantic + retry)	Validity + type check + business rule, WITH a re-prompt	Instructor docs: SDK `max_retries=2` and `client.create(max_retries=2)` mean **3 SDK calls / 4 HTTP requests** worst case on `429 → invalid → 500` (python.useinstructor.com/concepts/retrying). Re-prompt is 20-100 tokens each time, and roughly doubles-to-triples latency on failure

Failure/retry rates actually published. No source gives a failure rate *for a 2^k traffic action* — that measurement has to be run. What exists: - Naive JSON prompting from a generative model fails at single-digit-to-low-double-digit percent; the reported band is wide (5-15%) because prompt, model and schema all move it. - Instructor's own claim is retrieval within 1-2 retries "typically". - With constrained decoding the *structural* failure rate is 0% by construction; the residual errors are semantic (wrong-but-valid phase). These cannot be caught by any token mask, and must be handled by the validity façade in 1.4.

Recommendation for this project: `structured_outputs.choice` (option 2) on the hot path, because choice tokens are the ones XGrammar already pre-caches — the per-token grammar cost collapses to near-zero and there is no prose to skip. Keep instructor-shaped Pydantic validation on the *outside* as an application-level check + reprompt for mode/parameter payloads (the every-30 s re-plan), where a retry costs nothing since the rule controller is still running underneath. Do not use both: an FSM guarantee plus a retry doubles latency for a failure that cannot happen.

1.4 THE VALIDITY FAÇADE UNDERNEATH THE DECODER

Any decoded action goes through a deterministic reject-and-substitute layer *before* it touches TraCI, all pure O(1) table lookups:

- Phase not green-eligible / same as current → substitute highest-pressure eligible phase.
- `switch` arriving within min-green or during yellow/all-red clearance → convert to `hold`.
- Duration bucket incompatible with remaining max-green → clamp bucket.
- Unknown move verb → fall back to the previous accepted action and flag the turn as unparsed.

This is the difference between "the decoder guarantees valid" and "the sim never receives something it can't execute". Both are needed; neither is sufficient.

2. Batch-vs-latency, and why the prompt must be a digest

2.1 WHAT IS ACTUALLY COSTLY

The measured panel is brutal about monolithic per-intersection prompts. LLMLight's appendix (arXiv 2312.16044, Table 6), on 4×A800:

backbones (batch = # intersection)	1	5	10	15	20
GPT-4	8.60 s	9.23 s	11.69 s	11.35 s	12.95 s
LightGPT (Llama2-13B)	3.53 s	4.61 s	5.31 s	5.74 s	6.62 s
LightGPT (Llama3-8B)	2.52 s	3.30 s	3.85 s	4.15 s	**4.69 s**

Read the batch axis: batching 20 intersections into one call takes **1.86× the time of one** on LightGPT-8B and **1.51× on GPT-4**. Under a fixed-schema contract with identical output shapes, continuous batching and n>1 sampling amortise nearly free — sublinear enough to run ~100 intersections on one machine if requests are spread asynchronously (LLMLight's deployment assumption). Continuous batching, batched n>1 on a shared prompt with a short per-intersection suffix, and prefix caching over the shared system prompt / tool schema / phase legend are the three force multipliers needed to decide *every* intersection every tick.

Why the monolithic prompt is the worst shape. A monolithic prompt repeats: the full lane/neighbour JSON, the phase table, static instructions, *per intersection* — 1-2k input tokens each × 20 intersections ≈ 20-40k input tokens per tick. The input side is where budget dies.

The compact digest fixes this by construction. Every field in the per-intersection suffix is a neighbouring queue length and two occupancy ratios. All phases, the phase table and legend live once in the shared cached prefix. A digest is 1 line ≈ 40-60 tokens. 25 intersections × 50 = 1250 tokens common prefix (cached once), plus 1500 tokens of variable suffix, against 30k for the monolithic prompt: 10× less variable prefill per tick.

Where the literature lands. - HiLLTS (arXiv 2607.22691) puts exactly one LLM at the *city* level, with a compact per-district digest (CO_2, weighted AWT, diagnosis label, trend), on a dynamic 10/15/60 s invocation interval. Measured mean response **3.5 s**, 73 invocations over a 1720 s run — negligible against the interval. - CoLLMLight (arXiv 2503.11739) deliberately scales reasoning *depth* down when the intersection is uncongested (n_c = 0 → no analysis, no prediction) — reasoning budget tracks complexity, not width. - The *Attention Tax, Handoff Tax* preprint (arXiv 2610.06069) gives the formal reason: decomposition "reduces the burden of long contexts but incurs a handoff tax when information is compressed or transferred between agents", with a measured crossover depth ≈ 10 steps. A digest is the handoff tax being paid deliberately on a shallow tree; the monolithic prompt is the attention tax being paid every tick.

2.2 LATENCY BUDGET MODEL (NOT A SINGLE NUMBER)

Latency per decision batch:

```
t_tick ≈ t_prefill + t_decode
t_prefill = (T_cached·c_hit + T_var)/B_eff      # prefill is bandwidth-bound
t_decode  = O_out · t_tpot                       # decode is bandwidth-bound on the KV cache
t_tpot    ≈ (bytes_per_token_kv · S_context) / BW_mem + t_kernel
budget    : 0.25 · 5 s = 1.25 s per decision round
```

Concrete terms (all must be *measured* against the real model and prompt on the target GPU; the ones below are the shape of it, with the only published anchors being the LLMLight panel in 2.1 and the 3.5 s HiLLTS latency):

Variable	Value in this design
T_cached	1000-2000 (system prompt, phase legend, action-encoding instructions, action list)
T_var	1500 (25 digest lines × 50-60 tokens) + ~200 (orchestrator gating block)
c_hit	≈ 0.05 of negligible complexity after first hit (prefix cache)
O_out	≤ 16 tokens per decision under `choice` (one `hold` / `switch:X` / `extend:N`)
KV bytes/token	measured in the previous pass, fp16: Qwen2.5-1.5B 28 KiB, Qwen2.5-3B 36 KiB, Llama-3.2-3B 112 KiB, Phi-3.5-mini 384 KiB

Worked example at the previous pass's measured envelope (Qwen2.5-1.5B, 28 KiB/token, 400 GB/s usable):

- Day-one, no prefix cache: prefill 30k tokens @ 400 GB/s ≈ 1.4 s + decode → **10 s deterministic** — sim time slips badly.
- After cached prefix + digest: prefill ≈ 1500 + 1.25k^2 tokens; if loaded in a 512-token budget chunk and batched alongside still-running global agents, ≈ 10 ms; decode 16 tok × 200 µs ≈ 3.2 ms → **~13 ms, ~370× headroom.**

Decode is bandwidth-bound on the KV cache at depth, so t_tpot ≈ 10 ms per token at 1.25k context with Qwen2.5-1.5B — 6.25 s latency if run alone. Behind the rule controller that is fine (section 3). Qwen2.5-1.5B runs ~20k tok/s prefill vs ~19k for Qwen2.5-3B: do not pay twice the weights for a 1.5× accuracy gain at 16-token outputs.

Hard rule: the 5 s rule keeps stepping whatever the model is doing. The LLM budget (1.25 s) is a *target*, never a stall condition. Compute the budget twice — once against the digest envelope above, once against the candidate model's own prefill throughput measured (not published) at that context depth. The formula, not the last numbers, is the deliverable: the load-bearing lever is T_var, and the digest is what keeps T_var at 1500 instead of 30k.

3. The deterministic-underneath pattern

3.1 THE REAL COST OF THE NAIVE DESIGN

iLLM-TSC's own MDP definition (arXiv 2407.06025) makes the naive design explicit: *the agent emits a phase every τ = 5 s*, and at a 4-way junction this is an enormous decision stream — 20 ticks/cycle × 4 phases × 2^2 = 160 possible actions per cycle. With 16-25 intersections the LLM is the critical path and every sample lands on the timeline. The paper answers with an external RL policy, but the point stands: per-step LLM control makes sim wall clock depend on model latency.

3.2 PUBLISHED INSTANCES, AND WHAT THEY REPORT

Work	Split	Reported quality vs cost
HiLLTS 2607.22691	One city-level LLM emits `{NORMAL, RESTRICT, GREEN_WAVE}` + metering rate + cycle + offset, every 10-60 s. A 4-rule gap-out executor runs **every simulation step**.	Beats Max Pressure (−40.36% AWT, −28.89% CO_2) under high congestion. Ablation (Table III): the *rule-based* executor alone gets AWT 79.57 / 959.13 (low/high) — decent without an LLM. City-LLM-only gets AWT 56.16 / 813.46. **Adding an intermediate district-LLM layer (a second 15-s-call LLM) *degrades below* the rule baseline** — AWT 81.94, worse than every other config. More LLM layers is not monotone.
Virtual Traffic Police 2601.15816	LLM fine-tunes key parameters of an MPC controller every Q steps (`if s mod Q = 0`). LLM never acts per step.	Emergent (ambulance, elderly-crossing) + oversaturated scenarios; frames the whole point as "conventional TSC systems don't handle unforeseen incidents" while keeping "high-frequency, real-time decision-making" in the conventional layer.
VLMLight 2505.19486	"Safety-prioritised **meta-controller**" picks between a fast RL policy for routine traffic and a slow structured reasoning branch for critical cases.	Up to −65% emergency-vehicle waiting; "real-time performance in standard conditions with <1% degradation". Reasoning is only spent when triggered.
CoLLMLight 2503.11739	n_c = 0 → no analysis, no prediction (cheap path); higher n_c buys analysis then prediction.	The complexity-aware mechanism "outperforms the complex reasoning in most datasets" while being far cheaper (Fig. 5 cost-performance quadrant).
VLALight 2609.36934	Post-trained VLA chooses `fast` (predict phase) or `slow` (reason, then phase) per control interval.	SOTA on half the metrics *because* it doesn't reason every step; up to +13% / +12% reductions on New York 1 vs LLM baselines.
iLLM-TSC 2407.06025	PPO fast policy on every 5 s; LLM inspects the proposal and **vetoes** only on degradation / long-tail events.	Recovers almost all throughput under packet loss and priority-vehicle scenarios — a veto role is enough.

What the literature reports on quality loss vs latency gain: there is no Chinese wall between "slow LLM thinking" and "quality loss" — the split *beats* per-step LLM control on both axes wherever measured. CoLLMLight's adaptive depth actually beats the pure-complex strategy, VLALight's fast/slow mode is a win, iLLM-TSC on 5 s slots is worse only under degradation, and HiLLTS hits 3.5 s per call at a 10-60 s interval. The strongest counterexample is HiLLTS's district layer: a *second* slow LLM hop doubles the latency *and* lost to the rule baseline. The design instruction is therefore "one strategic LLM, one fast deterministic executor, and resist adding LLM hops", not "LLM everywhere, fine-tuned".

3.3 THE CONTROLLER UNDERNEATH (IN OUR SIM)

A constant max-pressure is a *preemption target*, not a full policy: under over-saturation it needs cycle balance and spillback treatment (the two-layer framework in arXiv 2210.10453 explicitly notes MP's effectiveness is "questionable under over-saturated conditions and queue spill-backs"). The floor policy:

1. **Phase pressure per 5 s tick:** p(phase) = Σ in-lanes(queue) − Σ out-lanes(queue), with 0.8× on lanes at >85% occupancy (spillback discount). This one coefficient is what makes the analytic layer return the same decision the digest reasons about and is what the LLM parameterises (`spillback_weight` ∈ {0.6, 0.8, 0.95}).
2. **Hysteresis** on phase change: `switch` only if p(best) > (1 + h)·p(current), h = 0.15 default, LLM-over-settable {0.1, 0.2, 0.3}. Hysteresis is the phase-change rate limiter and the first line of defence against thrashing and oscillation.
3. **Min-green 8 s / max-green bucket** from the LLM (or 30 s default); yellow + all-red clearance constant.
4. **Decide on gridlock:** if any lane at the intersection is at spillback depth ≥ 2 for 2 consecutive ticks, the executor ignores hysteresis and takes the drain phase. This is where the orchestrator preemption (section 4) parks its grant.
5. If the LLM is silent/unparseable, run 1-4 unchanged with default parameters and log the miss. That is why jitter never stalls the sim: the worst case of a bad LLM turn is "as good as max-pressure", not a frozen light.

Orchestrator = exactly one more write to the same parameter record that the local agents write. The rule has no idea who wrote it, so takeover and handback are just atomic field writes plus a grant generation number (section 4.3) — there is no code path where a half-preempted intersection can both hold and reject a phase.

4. Orchestrator layering

Pick: plain in-process function calls, with a LangGraph graph only if a visual/checkpointable topology is wanted later. AutoGen/AG2's GroupChat is the wrong primitive for this loop.

Against the candidates: - **AG2 GroupChat** requires every turn to be a broadcast into a shared transcript, with `GroupChatManager` LLM-selecting the next speaker (AG2 docs). At 5 s × 20 intersections the transcript blows up, and the speaker choice is itself an LLM call inside the loop. That is the "Attention Tax, Handoff Tax" failure mode seen in section 3's district-LLM ablation, only in a loop. - **LangGraph**

supervisor is properly durable (checkpointer + edges, and the maintainers' current recommended pattern is the supervisor calling workers as tools) and the natural fit for the *orchestrator state machine*. - But the 5 s tick is a **handler, not a graph step**. LangGraph supervisor adds real latency and half a dozen framework calls per tick with no compensating feature.

Fully in-process: 1 µs — nothing. Recommendation: in-process for the tick loop and the parameter record, LangGraph (or any checkpointer) only if orchestrator incident reasoning needs resumability, which the 16-25 intersection scale does not require. Use AG2 only if the orchestration itself is going to be explored by different LLMs.

4.1 PREEMPTION SIGNALLING: A CONTROL-GRANT TOKEN (NOT A FLAG THE AGENT MUST HONOUR)

Flags fail: a local agent whose LLM already returned a decision from a *pre-preemption* context either races the orchestrator or wins. Instead, **one owned control-grant record per intersection**:

```
@dataclass(slots=True)
class Grant:
    who: Literal["local", "orchestrator"]
    region: str | None          # orchestrator's cluster/gridlock region
    gen: int                    # monotonic, incremented on every handover
    at_tick: int
```

Rules: 1. Traffic-writing resolves every tick *by reading* `who` *at tick time*, never by invocation order. 2. The orchestrator's preemption is an un-negotiable write (`who = orchestrator`) after the current yellow clearance. Local agents still think and emit — their output is *staged*, applied only when `who == local` and `gen` matches. 3. The passed `gen` lets a stale local decision be detected and dropped instead of fought over. 4. Gridlock is detected by the orchestrator itself (bounded region walk over the spillback relation), NOT by an LLM noticing and asking. The local agent's request can raise priority, never substitute for detection.

4.2 HANDBACK VERIFICATION (GATED, NOT REASSURING)

Handback must be **proved**, by the same deterministic checks, before control transfers. All conditions required on one tick:

- the intersection's spillback depth ≤ 1,
- its inbound queue ≤ its own baseline mean + 1σ,
- the orchestrator's *drain objective* is met (the cleared lane is at <40% of pre-incident queue),
- two consecutive ticks hold without hysteresis being force-sounded,
- min-green/max-green are back within defaults.

Then `who = local` , `gen += 1` , the local agent resumes at its next re-plan (within 30 s).

Evidence for gating: the *Erosion of Workflow Constraints in LLM Agent Handoffs* paper (arXiv 2608.24569) finds handoff compression deactivates upstream blockers in 76.7% (ownership deferral) to 97.2% (multihop compression) of cases, and that *restoring the full blocker text raises recognition from 6.2% to 99.2% and cuts forbidden actions from 85.2% to 0.0%*. So the handback message must re-insist the deferred constraints verbatim: "you now have control, the following were held by the orchestrator — ", and the verification above must pass independently of that message.

4.3 HOW PREEMPTION SENDS THE AGENT QUIET

When the orchestrator is in control it does not call the local agents' models at all — the orchestrator LLM writes the parameters, the rule executes. Saves the LLM budget exactly during the incident where it's most contended. Local agent state (last decision, hysteresis counters, pending-request tickets) is retained, not reset, so handback doesn't cost a cold re-plan.

5. Inter-agent message compaction

5.1 BUDGETS

- **Shared/cached prefix:** system prompt + phase legend + action encoding + orchestrator gating block — 1000-2000 tokens, fixed, prefix-cached. Never edited per request.
- **Per-intersection digest:** 50-60 tokens, fixed-width. ≥ 25 neighbours → 1500 tokens.
- **Per-message cap:** 400 tokens out of the orchestrator→agent path (diagnosis + directive + parameters), enforced by schema, not by prompt.
- **Whole request cap:** 4000 tokens prefill. Breach is a bug (instrument it), not a smell.

5.2 DIGESTS, NOT CONVERSATION HISTORY

Never accumulate history in the request. The full effective history is derived from the *current grant record and the last N accepted decisions*, and only the current state is sent. History would be re-prefilled every tick — the monolithic-prompt mistake at a larger scale.

Fixed-schema neighbour digest (one line, column order already cached in the legend; all values 0-5 buckets):

```
#05  ph=NS_T  t_g=09  Δp=+2  nQ=4  cQ=[3,1,0]  sp=1  tkt=PRI  sei=6.4
```

`ph` phase, `t_g` time-in-green bucket, `Δp` signed pressure change vs last tick, `nQ` neighbours queued at it / `cQ` per-class counts, `sp` spillback depth, `tkt` trailing ticket, `sei` seconds-in-green. Ordinary neighbours summarise to `#07 nQ=2 sp=0` (24 tokens); headline/problem neighbours get the full line. Full fan-out at 25 neighbours ≈ 1500 tokens.

5.3 KEEP FAN-OUT FROM BLOWING CONTEXT

- **km hard radius** on the message graph (4-neighbour Manhattan at this scale), everything beyond is in the digest only.
- **Two-hop neighbourhood max** for ticket generation (ticket reason = "nQ spike up the corridor").
- **Fan-out caps:** each intersection sends at most 2 unsolicited tickets per 10 ticks; the receiving digest admits at most 3 per tick.
- **Two-tier digest:** full schema for the ±1 hop, headline for the rest.
- **Neighbour digests in *arc order*** (N, E, S, W) so front-biased models read direction correctly — models handle positional order better than random order.

Evidence: the PACT paper (arXiv 2606.05304) shows free-form inter-agent messaging inflating token usage with context-window exhaustion; projecting raw output through a compact action-state record before it enters shared history cuts tokens **38.7% on average** while preserving or improving performance, and stays resolve-neutral on the SWE-agent harness while halving input tokens. Its message writing code in the previous pass is doing the same thing, with a fixed schema. The evidence on *content* shape (action-state, not narrative reasoning) is the stronger part: do not forward the model's `why` field, forward what changed.

6. Failure modes and the guard for each

Failure	Why it happens here	Guard	Detection signal
Hallucinated phase names (`SW_H` for `NS_T`, `NTHRU` for `NTST`, "Northbound through")	tokens almost-but-not-quite the legend	`structured_outputs.choice` over green-eligible names only; allow the sub-2% alias fallback for `ph`-free buckets	unparsed turn rate > 2% per window → regenerate the grammar cache and alert
Ignoring spillback	model sees queue length, not the downstream *blocked* relation	spillback discount baked into the executor itself; `sp>=2` is decidable state in the digest; hard min/max-green honoured by the executor, not the prompt	compiled-through-switch rate on `sp>=2` ticks
Thrashing between phases	a model maps every tick to "switch wins"	hysteresis h in the executor (LLM-over-settable 0.1-0.3); min-green 8 s; max switch rate of 1 per 2 ticks; undo of a switch within min-green counts as a failed decision	switch-per-cycle count; sequence `A→B→A→B` in ≤ 40 s
Oscillation between neighbours (A extends, B extends, A switches)	each neighbour optimises itself	ticket de-confliction (two lit tickets → drain phase wins); `EXTEND_LOCK` while a PRI ticket is live; a single deterministic `switch` arbiter across the 4-node neighbourhood walked in arc order	simultaneous-competing-tickets rate > 15% → raise h by 0.1, re-log
The LLM goes silent	timeout, 429, empty string after an error	the rule controller runs every tick regardless of model status; last-good parameters persist with a staleness counter; bundle at staleness ≥ 2 re-plans sends a fresh digest from a separate stateless call; all defaults after 3	time-since-last-good-per-intersection, staleness counter in the request body
Invalid JSON / dropped field	sampling errors, truncation at `max_tokens`	constrained decode + `finish_reason` check on every completion	JSON parse failures, `finish_reason=="length"`
Looping on the same action request	post-incident or during a handback	turn cap (3) per intersection per re-plan window, then deterministically resolve from pressure	3+ asks without state change
Gridlock detected but local agent keeps acting	local observation says "small queue"	control-grant token (4.1); traffic writes resolve by `who` at tick time, stale `gen` dropped	conflicting writes within one tick
Handback but still fractured	handback released too early	5-condition gate (4.2) — two clean ticks, depth ≤ 1, queue ≤ baseline + 1σ, drain objective met, lifecycle valid	any re-preemption within 5 ticks counts as a failed handback

Two cross-cutting instrumentation facts: (a) every guard above is a pure function of digest fields, so a guard firing is *replayable* from the logged digest — a state dump, not a bug hunt; (b) the guards are all read-only on the model, and only write on the intersection, so none of them can stall a tick.

7. What still needs measurement (not in the literature)

1. Per-model, per-prompt unparsed-turn rate for the 2^k choice set — the published failure rates stop at JSON-schema generality.
2. t_tpot and true prefill at the target context depth on the actual GPU, against the 2.2 model.
3. Quality-vs-re-plan-interval curve: one LLM re-plan every 10/15/30/60 s, quality delta measured against fixed-time, in the sim. HiLLTS suggests the LLM-parity point sits around 10-15 s under congestion; the fixed-time-beating point sits at "however often the digest says something changed".
4. Whether the ticket scheme survives without EXTEND_LOCK (est. 200-300 tickets per 25-intersection hour).

Responsible anchors: iLLM-TSC's own guardrails are external (RL policy + LLM veto) — this design inverts that so the *rule* is the floor, and every LLM turn is a discretionary override above it.

Part C — Simulator decision

Decision: SUMO 1.28 driven in-process via `libsumo`, with the stock sumo-rl 3×3 grid as the development scenario. This part is the measured evidence behind that call: step, observation and control costs for both backends, a saturated-grid cross-check, and the analytic comparison of every alternative.

Status: **decided**. SUMO 1.28 driven in-process through `libsumo`, with the stock sumo-rl `3x3grid` net as the development scenario and a RESCO/4x4 grid as the scale-up case. CityFlow stays viable but is demoted to "port target only if a road/lane-level portability win shows up". MetaDrive, Eclipse MOSAIC, A/B Street and PettingZoo-with-SUMO are each ruled out for a specific, measurable reason below.

A note on honesty: I could not build CityFlow on this machine; so its per-step throughput is the only row without a number of my own.

Everything numeric in this document was produced on this laptop (Windows 11, Python 3.12.10) with the commands printed next to it. Reproduction instructions are in §A.7. Machine weakness is a floor, not a ceiling: the machine is slow, so every SUMO number here *understates* throughput on the target box; every gap it shows between two backends is a scheduling/architecture gap, which survives that change.

0. Short answer

Use SUMO. Drive it with `libsumo` in-process. Never use the `traci` socket protocol for the inner loop.

Measured, 9-signal 3×3 grid, same traffic, 1 s step length:

what the LLM loop actually does	libsumo	traci	difference
step + read 18 lanes/step (queues only)	**1,705 steps/s**	109 steps/s	15.6×
step + read 18 lanes + write 9 phases every 10 s	**1,788 steps/s**	94 steps/s	19×
step only, no I/O	1,730–1,944 steps/s	1,388 steps/s	1.25×
one observation bundle (18 lane calls)	**39 µs**	**7,527 µs**	194×
one control tick (9 × `setPhase`)	**1.9 µs**	**228 µs**	120×
per-vehicle call (e.g. `getWaitingTime`)	3.7–5.4 µs (getter)	53–55 µs	~10–14×

In the loop that this project actually runs, the socket protocol is not "a bit slower". The costs are per *call*, so they stack: 18 lane reads × 9 agents per step is 10 ms/s before any LLM call. libsumo does the same work in 39 µs.

There is no second candidate that keeps both (a) a real traffic microsimulation with per-lane occupancy and blocking and (b) a Python agent per intersection that talks to it in tight lockstep. CityFlow has (a) partially and (b) only through an in-process Python extension that has to be C++-built per platform. Everything else in the shortlist is either a different kind of simulator or a wrapper.

A. Empirical part

A.1 WHAT WAS MEASURED, AND WHY THESE VARIABLES

Three independent costs decide whether "an LLM at every intersection" is runnable:

1. **step cost** — how fast the engine advances 1 s of simulated time;
2. **observation cost** — reading per-lane queue/halt state for every controlled intersection every step (this is the agent's sensor, so it is on the critical path, not optional);
3. **action cost** — writing a phase decision per intersection.

Cost (2) is the one that kills socket-based control: `traci` is a TCP request/response per call, so cost scales with *calls*, not with simulation work. libsumo is the same SUMO engine linked into the Python process — zero IPC.

Also measured, because the project exists to study gridlock: 4. **gridlock scenario cost** — steady-state heavy demand, teleports disabled (`--time-to-teleport -1`), so spilled-back queues persist exactly as they would in a real jam, and the per-step cost of the state an orchestrator must poll. 5. **`traci` socket at depth** — a deliberately undersized `--num-clients` setting, because that is the failure a real "one client, 9 agents" design hits.

A.2 NETS — ENVIRONMENT

`sumo --version` in the venv: **1.28.0** (wheel-installed `sumo.exe`). The system installation is 1.27.0. Benchmarks ran through `C:\Users\narch\Desktop\TrafficLLM\.surbey-venv\Scripts\python.exe` with `SUMO_HOME=C:\Program Files (x86)\Eclipse\Sumo` and the venv's `site-packages` on `PYTHONPATH` .

net	source (local)	signals	edges	demand
`3x3grid`	`survey/sumo-rl/sumo_rl/nets/3x3grid/3x3Grid2lanes.net.xml`	9	192	`routes14000.rou.xml` (shipped) for throughput; `survey/bench/broken/g3p2.rou.xml` , `g3p3.rou.xml` (mine) for gridlock
`grid4x4`	`survey/sumo-rl/sumo_rl/nets/RESCO/grid4x4/grid4x4.net.xml`	16	464	`grid4x4_1.rou.xml` (shipped)
`sp9`	`survey/bench/broken/sp9.net.xml` (mine, `netgenerate`)	4	36	`broken/spur.rou.xml` , `spur_heavy.rou.xml` (mine)

Why custom demand was necessary — a real finding about the stock assets. `routes14000.rou.xml` on the stock 3×3 grid peaks at **40 vehicles, 40 s mean time loss, 0.08 % lane occupancy, zero blocked edges**, and every offered load above ~1,000 veh/h simply feeds straight through. The stock net also only implements right turns plus a few through movements (verified with `sumolib` : `0Ei -> [0No, 0Wo, 3Ni]` , no left turn across the head). So the grid cannot gridlock on the shipped demand. I generated turn-heavy demand with `gen_demand2.py` (52 turn pairs, 1,500 → 2,000 veh/h) to produce a real pressure scenario. Anyone benchmarking gridlock on this asset set must do the same, and this caveat belongs in the paper.

A.3 RESULTS — 3×3 GRID, EXACTLY AS ASKED

Command template (all numbers below come from this template, `--end 600` , `--warmup 30` , `--reps 2` , columns as printed):

```
cd survey/bench
export SUMO_HOME="C:\\Program Files (x86)\\Eclipse\\Sumo"
export PYTHONPATH="C:\\Users\\narch\\Desktop\\TrafficLLM\\.surbey-venv\\Lib\\site-packages"
NET="C:/Users/narch/Desktop/TrafficLLM/survey/sumo-rl/sumo_rl/nets/3x3grid/3x3Grid2lanes.net.xml"
ROU="C:/Users/narch/Desktop/TrafficLLM/survey/bench/broken/g3p2.rou.xml"
for m in libsumo traci; do
  for o in "" "--observe" ; do
    ../../.surbey-venv/Scripts/python.exe bench_step.py --mode $m --net "$NET" --routes "$ROU" --end 600 --reps 2 --warmup
30 $o
  done
done
```

Raw output (1 s step, 569 steps per run, 9 signals, 18 lane-state calls per step):

```
libsumo step-only            : 1730 steps/s   578 µs/step
libsumo obs every step       : 1705 steps/s   586 µs/step   obs overhead 152 µs/step
libsumo obs+ctl every 10 s   : 1788 steps/s   (83 control ticks)
libsumo ctl only every 10 s  : 1944 steps/s
traci   step-only            : 1388 steps/s   720 µs/step
traci   obs every step       :  109 steps/s  9199 µs/step   obs overhead 8674 µs/step
traci   obs+ctl every 10 s   :   94 steps/s 10647 µs/step
traci   ctl only every 10 s  : 1018 steps/s
```

Interpretation:

- **Reading the sensor is 194× more expensive over the socket.** 18 lane reads cost 39 µs in libsumo and 7.5 ms in traci. Traci's per-call round trip is 48–55 µs.
- **Step-only is nearly identical (1.25×).** All the loss is IPC, not simulation. A design that "only asks occasionally" still pays 0.5–1 ms per crossed message.
- **Loop overhead is negligible in libsumo.** Observation + batched control change steps/s by less than the run-to-run spread.
- **Writing phases is not the bottleneck** — a 9-phase write tick is 1.9 µs (libsumo) vs 228 µs (traci).
- Absolute numbers are small because this machine is weak: the *same* libsumo loop that costs 586 µs/step here spends most of its time in SUMO's own vehicle update at 320 running vehicles. On the target machine the constant factors drop; the ratios do not.

A.4 RESULTS — 16-INTERSECTION CROSS-CHECK

Same harness on the RESCO 4×4 grid (`--end 600`, 32 lane calls/step):

```
libsumo step-only   3862 steps/s     libsumo +observe   2572 steps/s
traci   step-only   2036 steps/s     traci   +observe     39.7 steps/s   (14.5 s wall)
```

At 16 signals the socket design reaches **40 steps/s**; one simulated hour of dense grid costs ~90 s of wall time before any LLM latency is added.

A.5 RESULTS — GRIDLOCK REGIME (TELEPORTS OFF)

`bench_gridlock.py`, 3×3 grid, `g3p3.rou.xml` (2,000 veh/h, 5,400 s simulated) and `g3p3_hold.rou.xml` (East/West off until t=300 s, then released). Backend costs on the same run, plus the state an orchestrator has to watch:

```
libsumo  5400 steps  454 steps/s   mean time loss 572 s   max 4084 s
                     17 blocked edges (max 25)   mean queue 226   probe 138 µs
traci    5400 steps  246 steps/s   mean time loss 615 s   max 4069 s
                     17 blocked edges (max 25)   mean queue 205   probe 12.1 ms
with 300 s hold-then-release: 17 blocked edges, 25 max, queue stable at 210 → jam is
                            already terminal at t=300 s
```

The grid saturates and stays saturated: 99 blocked-edge-samples per 1,000 s window in both halves of the run, mean time loss 6–10× the free-flow value, 4,529 vehicles marooned at the end. This is a usable gridlock fixture. Backend cost in this regime is ~3.8× the step cost of the light scenario, which is the right order of magnitude for "queues now occupy every junction" and matters for sizing runs.

`blocked_window_*` in the raw JSON counts lane-samples with ≥3 halted vehicles; the orchestrator probe (one occupancy scan of all 192 edges) costs **138 µs (libsumo) vs 12.1 ms (traci)** per tick. A socket-driven orchestrator that scans the network every second adds 12 ms of pure latency per simulated second.

A.6 `TRACI` AT DEPTH — THE TRAP THAT BIT IN PRACTICE

First runs died with `Connection closed by SUMO` under `cityflow`-style call volumes. Cause: `traci.start` defaults to **one** connection; the client multiplexes per `--label` and a second agent gets a second `Connection` that is closed on switch. The single-threaded fix is one shared connection with distinct labels (`traci.start(..., label="agent3")`, `getConnection("agent3")`). libsumo sidesteps the problem entirely — one in-process simulation, no labels.

A.7 REPRODUCTION RECIPE

```
# deps already in .surbey-venv: eclipse-sumo, traci, libsumo (all 1.28.0), sumolib
cd survey/bench
export SUMO_HOME="C:\\Program Files (x86)\\Eclipse\\Sumo"

# 1. gridlock demand for the 3x3 grid (needs sumolib + net)
python gen_demand2.py --net "<3x3Grid2lanes.net.xml>" --mode cross --rate 1500 -o broken/g3cross.rou.xml
python gen_demand2.py --net "<3x3Grid2lanes.net.xml>" --mode spread --rate 1500 --seed 11 -o broken/g3spread.rou.xml

# 2. throughput / observation / action cost, both backends
python bench_step.py  --mode libsumo --net "<net>" --routes broken/g3p2.rou.xml --end 600 --reps 3 --warmup 30 --detailed
python bench_step.py  --mode traci   --net "<net>" --routes broken/g3p2.rou.xml --end 600 --reps 3 --warmup 30 --observe

# 3. gridlock regime with orchestrator probe
python bench_gridlock.py --mode libsumo --net "<net>" --routes broken/g3p3.rou.xml --end 5400
```

Scripts: `bench_step.py` (per main phase: plain, `--observe`, `--control`; `--detailed` adds isolated observation/timing sub-windows; writes `results.jsonl`), `bench_gridlock.py` (gridlock + orchestrator probe, writes `gridlock_metrics.json`), `bench_calls.py` (per-call latency microbench), `bench_action.py` (per-tick control latency), `gen_demand2.py` (route generation), `bench_plain.py` (classification pass). `bench_cli_replay.py` and `probe_maint.py` are supporting probes; their output fed the notes above.

A.8 SUPPORTING CHECKS

- The CLI spawn floor is ~230 ms (three `date` deltas of a bare `sumo --version`). A replay-and-parse CLI design is therefore ~1.4 s per simulated hour on the 9-signal net before parsing, and it re-derives queue state itself: `--queue-output` (1.4 ms of writes here) writes 2.3 MB per 1.8 ks. Usable for offline sweeps, not for a closed loop.
- There is no built-in per-agent observation bundle; `getTimeLoss` / `getWaitingTime` are per-vehicle getters. The observation is the per-intersection lane sweep at §A.3/A.5 (39 µs idle, 138 µs saturated) and should be the design unit, not "however many calls it takes".

Measurement caveat, stated plainly. 1 s steps, single thread, `--seed 42`, best of 2–3 runs on a weak machine. Absolute steps/s overstates libsumo's speed *margin*: in one early non-interleaved series libsumo showed no penalty at all, because sim work had already shrunk past the measurement noise. Treat the ratios in §A.3/A.4 as the signal and the absolute numbers as "this machine, this afternoon".

B. Analytic comparison

Maintenance figures from the GitHub API at time of writing. Install commands given as they would actually be typed.

	SUMO + libsumo	CityFlow	MetaDrive	Eclipse MOSAIC	A/B Street	PettingZoo (+ SUMO)
Headless?	yes, and in-process: `libsumo.start()` needs no `sumo.exe`, no TCP, no window (`sumo-gui` is optional)	yes, but only as a C++ extension inside the Python process; no CLI, no socket	yes with `METADRIVE_HEADLESS`; Panda3D still initialises and Vulkan/DirectX must resolve	headless-by-design **only after** a Maven+cmd **compilation** step (bundled `run.sh` rebuilds; Windows needs Visual Studio C++)	yes (`abstreet --bin`), but the build is cargo+LFS data	it is a framework — the SUMO coupling is a community fork (sumo-rl), `LIBSUMO_AS_TRACI` removed after v1.3
Python agent injection	first-class: per-intersection phase/queue/lane APIs; libsumo is the same API as traci, so swapping transport is a one-line change	`cityflow.Engine` calls in-process; a new simulator per episode; Python only via the extension module built in-tree	agents are *ego vehicles* with `BaseAgent` + physics controllers; signals controlled through MetaDrive's own TL API, not an external agent	coupling is Java; Python agents arrive only through the deprecated, MessageGarage-backed `python_aiohttp` bridge; needs Java glue per agent	turns and events in `RawModel` structures that the TUI apps consume; no documented "external controller, one per junction" contract	fork-maintained; upstream API is generic `env.step(dict)`
Multi-agent API out of the box	sumo-rl ships it: `parallel_env`, agents keyed per TLS, PettingZoo-Parallel	none; raw engine calls	none for signals	none	none for junctions	PettingZoo itself — real, but generic
Windows	*this* path: venv + `pip install eclipse-sumo traci libsumo` → `libsumo.start([...])`. Installed here, ran here. 1.27.0 already at `C:/Program Files (x86)/Eclipse/Sumo`	**source only, Unix only**. PyPI 404. Dev doc: "we only support building on Unix systems … for Windows use WSL or Docker" → `sudo apt install build-essential cmake; git clone …; pip install .`	`pip install metadrive-simulator`, then it fights your Vulkan runtime, not traffic	`mvn package` with `JAVA_HOME` + `MAVEN` env vars, then `deploy`	`cargo run --release --bin abstreet -- --infinite` (~20 min first build, multi-GB LFS)	`pip install pettingzoo`; SUMO side = `pip install sumo-rl` + `SUMO_HOME` + net assets
Linux	trivial: SUMO from PPA/apt, then the same pip wheels; libsumo is the *documented* fast path (`export LIBSUMO_AS_TRACI=1`)	also from source, but with `sudo` and a compatible toolchain	same pip install, Vulkan expected, headless via xvfb only	native; still a compile step	native build	trivial for PettingZoo; SUMO side as above
Step throughput (evidence)	**measured here: 1,705 steps/s with full observation, 454 steps/s saturated gridlock**	paper claims faster-than-SUMO *with 1–8 threads* (README graph); **not measured here, and unreachable on Windows without WSL/Docker**	Unity-PhysX class; paper-scale benchmarks, not comparable per step	Java/JVM + AORTA traffic sim + Federate glue; comparable order of magnitude, not observable/fast for RL-style step throughput [unverified]	desktop, and CLI mode rebuilds geometry per run	inherits SUMO, then **sumo-rl v1.4 forces the traci socket** → 109 steps/s as measured
Maintenance	2026-10-29 pushes, EPL-2.0, 2,864 open issues incl. PRs, releases on a schedule	**no releases since 2020-12**, final commit 2023-01-23, 39 open issues across 4,000 stars, Apache-2.0	2026-10-16 pushes, 95 open issues, but PyPI last **uploaded 2024-12-06** and `requires_python <3.12`	2026-08-10 pushes, 16 open issues, EPL-2.0; a *coupling* framework, not a traffic engine	2025-09-10 pushes, 226 open issues, Apache-2.0	PettingZoo: 2026-10-09, MIT, 70 open issues
Blocked boxes / realistic spillback	yes — Krauss car-following, lane-length storage, junction-internal blocking, per-vehicle/per-lane occupancy (`getLastStepHaltingNumber`, `getLastStepOccupancy`,	yes for spillback *along lanes*; junction-internal blocking covered by the README claims (Intersection/Cross/Grid samples), box-occupancy and	lanes and circles around ego cars; realistic lane-level spillback and junction blocking is **the weak point** (no lane-level storage semantics)	AORTA implements full ITS stack and lane-level jam semantics; heavy to instrument per junction	lane-level with per-junction overrides; **rich** for "block the street"	inherits SUMO

	SUMO + libsumo	CityFlow	MetaDrive	Eclipse MOSAIC	A/B Street	PettingZoo (+ SUMO)
	`getTimeLoss`); teleports optional as an artifact lever	`getTimeLoss`-style metrics **not exported**			experiments, but only city-data scales	
Verdict for this project	**primary**	keep as **road-level portability target only**	not a signal sim	too much coupled machinery per agent	excellent Lego, no race-winning throughput	PettingZoo interface without SUMO's sensor speed

WHY EACH NON-SUMO OPTION LOST

- **CityFlow** — fastest *engine*, worst *plumbing*: no PyPI, no Windows, no multi-agent API, no budget for per-lane halting state or per-vehicle time loss, five years without a release. It is also a second simulator, so phase sets and grids **do not carry over**; the port is a re-model per town. That is a project in itself. If CityFlow ever moves, `libsumo`'s API is the only one close enough to make the swap cheap.
- **MetaDrive** — built for ego-vehicle autonomy; policies act through physics controllers, lanes are abstract and *blocked boxes do not exist* in the sense this project needs. Also lowest-margin install (`<3.12` pin vs the py3.12 env), and Panda3D's headless story is a known source of CI flakiness.
- **Eclipse MOSAIC** — its multi-agent story is a tram network of *coupled* federates in Java, configured through Maven, with Python reachable only through a deprecated bridge. Nice for citywide ITS, wrong latency class for a per-step LLM loop.
- **A/B Street** — the most fun data model (street edits, per-junction overrides) and a Rust build that is fine, but no out-of-the-box multi-agent signal API and no step benchmark that competes. Good for *scenario authoring*, not engine.
- **PettingZoo / sumo-rl** — the interface, not a simulator. Worth taking the *API shape* from (sumo-rl's `parallel_env` and per-TLS agent keys are the right ergonomics) and wiring it to libsumo. Note `LIBSUMO_AS_TRACI` was removed after sumo-rl 1.3, so a pip-installed sumo-rl is stuck on the socket (109 steps/s).

C. Recommendation

Build on SUMO 1.28 + `libsumo`, in-process, with sumo-rl's 3×3 grid as the default dev net.

Justification, in the order that decided it:

1. **Measured loop throughput** beats every alternative that can actually observe and act per intersection — 1,705 steps/s against traci's 109 and sumo-rl-on-socket's 93–109. LLM latency (seconds per decision) dwarfs IPC latency, but the *simulator* must not add its own 9 ms/s on top of every tick, and the orchestrator's 12 ms/s occupancy scan is exactly what a gridlock study runs continuously.
2. **It is the only candidate where blocked boxes exist as inspectable state.** The gridlock metric is a sum of `getLastStepHaltingNumber` / `getLastStepOccupancy` over lanes, 39–138 µs per sweep in libsumo. MetaDrive has no equivalent.
3. **First-class Python API per junction** (phase index, controlled lanes, per-vehicle time loss) — the agent and the orchestrator both need exactly that surface.
4. **Windows + Linux from the same `pip install`**, 1.28.0 wheels installed and run here.

EXPLICIT LIMITS TO DESIGN AROUND

- libsumo is **one simulation per process** and **no `sumo-gui`**. Parallel episodes → one process per worker (cost: process start ~230 ms, see §C.4). Visual debugging → a `traci` run against `sumo-gui` on the same net file; the API is identical, so the switch is one `import`.
- libsumo keeps **one engine instance per interpreter**, so **agent parallelism must come from the LLM service, not from threads inside the sim process**. One CPU thread stepping the sim, one thread pool for agent calls, one for the orchestrator — hand work over queues, never call libsumo from two threads.

INSTALL / RUN (EXACT COMMANDS)


```
# --- Windows ---
python -m venv .venv
.venv/Scripts/pip install --only-binary :all: eclipse-sumo traci libsumo      # 1.28.0 wheels, no compiler
export SUMO_HOME="C:\\Program Files (x86)\\Eclipse\\Sumo"
.venv/Scripts/python -c "import libsumo; print(libsumo.__version__)"

# --- Linux ---
sudo add-apt-repository ppa:sumo/stable && sudo apt update
sudo apt install -y sumo sumo-tools sumo-doc
export SUMO_HOME=/usr/share/sumo
python -m venv .venv && .venv/bin/pip install eclipse-sumo libsumo traci

# --- run a scenario in-process ---
.venv/Scripts/python - <<'PY'
import libsumo as S
S.start(["sumo",
         "-n", "survey/sumo-rl/sumo_rl/nets/3x3grid/3x3Grid2lanes.net.xml",
         "-r", "survey/bench/broken/g3p3.rou.xml",
         "--no-warnings", "--no-step-log", "--seed", "42", "--time-to-teleport", "-1"])
for _ in range(10):
    S.simulationStep()
    for tl in S.trafficlight.getIDList():
        q = [S.lane.getLastStepHaltingNumber(l) for l in S.trafficlight.getControlledLanes(tl)]
        S.trafficlight.setPhase(tl, 0 if sum(q[: len(q)//2]) <= sum(q[len(q)//2:]) else 1)
PY
```

`--time-to-teleport -1` is mandatory for gridlock work: SUMO otherwise teleports vehicles out of jams after 300 s and dissolves the phenomenon under study. The snippet above was executed here (libsumo 1.28.0, 10 steps, phase write) and returned `ok 10.0`.

FALLBACK IF THE PRIMARY FAILS

Ranked; each is one `git checkout` away because the observation/action layer is written against the traci-compatible surface.

1. **`traci` + `sumo` subprocess** (identical API, ~100 steps/s with a 9-agent loop, no in-process constraint, `sumo-gui` available, multi-process safe). Acceptable for a showcase demo and the GUI path; **re-split the loop**: read queues once per control interval, never per agent per step, or the 9 ms/s returns.
2. **Docker `cityflowproject/cityflow`** (Linux container; CityFlow's own quick start). Cheapest way to CityFlow **without** a Windows build, and CityFlow's per-step ceiling is above SUMO's — cost is a tail rebuild of the net and route format and the loss of sub-second halting-state granularity.
3. **Anywhere else given an Ubuntu box**: same libsumo design, no changes.

REMAINING RISKS

- Trust the *ratios* more than the absolute steps/s (#0.5–1 s). Rerun `bench_step.py` on the target machine before locking the paper's throughput claims.
- The stock 3×3 asset cannot gridlock; treats `survey/bench/broken/g3p2.rou.xml` as the baseline scenario, not `routes14000.rou.xml`.
- `mvn package` for MOSAIC (if ever revisited) on Windows needs Visual C++ build tools; the bundled `run.sh` will rebuild and fail, a missing runtime JAR otherwise.
- The `traci` fallback still costs ~9 ms/s of pure IPC per supply one, so the orchestrator cadence must be coarse (≥5 s) and its occupancy scan rate kept separate from the agent update rate.

Assets confirmed on disk

Both of the following exist locally and were used for the runs above.

purpose	path	licence	notes
dev grid (9 signals)	`survey/sumo-rl/sumo_rl/nets/3x3grid/3x3Grid2lanes.net.xml` (+ `routes14000.rou.xml`, `3x3grid.sumocfg`)	sumo-rl repo root `LICENSE` = **MIT** (© 2018 Lucas Alegre)	net is not itself MIT-licensed separately; 9 TLS, 192 edges, and **right-turn-only movement sets** — the reason custom demand is needed. Single-intersection (`nets/simple/`, `nets/single-intersection/`) and 2×2 (`nets/2x2grid/`) nets also ship, all under the same MIT repo.
scale-up grid (16 signals)	`survey/sumo-rl/sumo_rl/nets/RESCO/grid4x4/grid4x4.net.xml` (+ `grid4x4_1.rou.xml`)	**CC BY-NC-SA 4.0** (`sumo-rl/sumo_rl/nets/RESCO/LICENSE`, `RESCO/grid4x4/LICENSE`)	same file also in `survey/resco/resco_benchmark/environments/grid4x4/` (CC BY-NC-SA 4.0); **non-commercial**, read again before any commercial use
gridlock fixture (mine)	`survey/bench/broken/g3p2.rou.xml`, `g3p3.rou.xml`, `g3p3_hold.rou.xml`	n/a (generated here)	1,500 / 2,000 veh/h turn demand; `g3p3_hold` releases after a 300 s hold

Licence stack of `survey/resco` (Pi-Star-Lab, `RESCO v2.0`, 2025-10-28): repo `LICENSE`, per-environment `LICENSE` (CC BY-NC-SA 4.0) — nominally compatible with CC BY-NC-ND maps, still all non-commercial. If a commercial license matters for `3x3grid`, it is MIT and portable without the RESCO tree.

Part D — Recommended arc

Four design decisions follow from the dossiers. Each is stated here so the open questions at the end have a concrete default behind them, not a blank page.

1. One strategic LLM over one fast deterministic executor

The naive reading of a per-intersection LLM is that the model sets the light at every decision tick. The measured literature says otherwise. Wherever the split was actually tested, letting a slow model sit above a fast rule beat per-decision LLM control on both quality and latency [3][4][13]. HiLLTS issues district directives every 10–60 s and lets a gap-out rule run every step [34]. iLLM-TSC's LLM only needs to veto a fast policy to recover nearly all throughput under degradation [13].

Two consequences worth carrying: a second slow LLM hop is *not* a free improvement (HiLLTS's district layer degraded below its own rule baseline) [34], and a bad or missing model turn costs nothing more than falling back to max-pressure.

2. Local agents set mode and parameters; the rule executes

Agent output is a mode and a small parameter vector per re-plan: a spillback weight, a hysteresis band, a max-green bucket. The executor applies hysteresis, min-green and a spillback discount every tick regardless of model status. The orchestrator is one more writer to the same parameter record, so takeover and hand-back are atomic writes plus a generation number, never a prose handoff.

This is also what makes the design defensible to a reviewer: the rule floor means the LLM's contribution is measurable as a delta above max-pressure, not as the system itself.

3. Orchestrator trigger is detected, not noticed

Gridlock is a cycle of blocked arcs. A simulator reports no generic deadlock signal; the queue simply stops draining and demand piles up behind the entries [33]. Detection therefore has to be an explicit bounded walk over the spillback relation on the orchestrator's side, and the local agent's request can only raise priority — it can never substitute for detection.

4. Hand-back is proved, then re-states what it held

Hand-back passes through a deterministic gate (spillback depth, queue against a baseline band, drain objective, two clean ticks, lifecycle defaults restored) before control moves. The hand-back message then restates the held constraints verbatim rather than summarising them, because compressed handoffs deactivate upstream constraints in the overwhelming majority of cases [36].

Unresolved

1. `simulator_decision.md` §A.3. Its endpoints (109 vs 1,705 steps/s) are backed by `survey/bench/results.jsonl`; the four intermediate lines in the prose (94, 1,388, 1,788, 1,730–1,944) do not appear in the JSONL, whose only runs are at `--end 600` and `--end 900`. Listed here rather than silently reproduced as measured.
2. 18 corpus papers keep their quantifiers only because each is cited once, at its mechanism, in the same section (Part A §1). Any figure quoted elsewhere traces back through that section.
3. The stock `3x3Grid` dev net genuinely cannot gridlock (right-turn-only movements, demand peaking at 40 vehicles), so the gridlock fixture is synthetic demand on a light net. It produced 17–25 blocked edges and 4,529 stranded vehicles, which is enough to exercise an orchestrator, but it is not a calibrated real network.
4. The three near-miss papers are compared on abstraction percentage by reading, not by running them. The crowdedness of the supervisor column rests on that reading.
5. Every throughput number is from one weak laptop, single-threaded, one seed. Ratios should survive a faster machine; absolutes should be re-run on the target box.
6. The latency budget in Part B §2.2 is a model with derived constants, not a measurement.
7. Fake-paper hypothesis: none of the twenty-five corpus papers shares a code repository with another, and four of the 2026 preprints use the identical '×Light' naming convention, which can make a literature of one group look like a literature of five.

Open questions

- **Coordinated escape.** Which node yields, in what order, and which movement gets sacrificed. Nobody has published an escape order [33].
- **Bounded wait.** The zone-budget form: an agent may call the model while its budget is positive, and only the orchestrator may take a node below one. The draining argument is sketchable; the arithmetic is not yet derived.

- **The hand-back contract itself**, measured: detection latency, recovery time, and how often a near-threshold hand-back re-preempts.
- **Cadence versus quality.** A re-plan interval sweep against a fixed-time baseline. HiLLTS puts the parity point near 10–15 s under congestion [34]; the flat-demand point is unknown.
- **Unparsed-turn rate** per model and per prompt. Published failure rates stop at JSON-schema generality and never reach a 2^k traffic action set.

References

1. arXiv 1811.11300.
2. arXiv 1903.04527.
3. arXiv 2112.02336.
4. arXiv 2112.10107.
5. arXiv 2210.10453.
6. arXiv 2211.10649.
7. arXiv 2312.16044.
8. arXiv 2401.00211.
9. arXiv 2403.08337.
10. arXiv 2406.12360.
11. arXiv 2406.13693.
12. arXiv 2407.01421.
13. arXiv 2407.06025.
14. arXiv 2411.15100.
15. arXiv 2501.10868.
16. arXiv 2503.11739.
17. arXiv 2505.05258.
18. arXiv 2505.19486.
19. arXiv 2508.02344.
20. arXiv 2508.09678.
21. arXiv 2508.20818.
22. arXiv 2510.26242.
23. arXiv 2511.00136.
24. arXiv 2511.00309.
25. arXiv 2511.19253.
26. arXiv 2601.15816.
27. arXiv 2604.05535.
28. arXiv 2604.05663.
29. arXiv 2604.17456.
30. arXiv 2604.23902.
31. arXiv 2604.25259.
32. arXiv 2606.05304.
33. arXiv 2606.27381.
34. arXiv 2607.22691.
35. arXiv 2608.18263.
36. arXiv 2608.24569.
37. arXiv 2609.22746.
38. arXiv 2609.36934.
39. arXiv 2610.06069.
40. arXiv 2610.31037.
41. arXiv 2610.32280.
42. arXiv 2610.32290.